

Lecture 5: Data Representation IV — Integer Arithmetic & Overflow

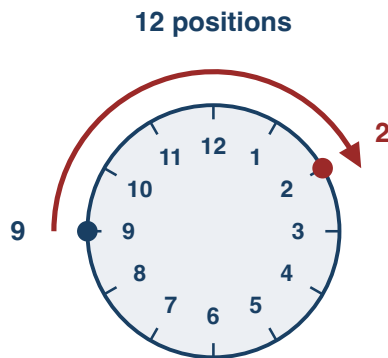
Session 5 · Mon Sep 14, 2026

Contents

Overflow Is a Clock, Not a Crash	2
Where We Are	2
Rewind: The Box From Lecture 2	2
The Top Bit Is Worth -128	3
The Odometer Rolls Over	3
The Size of the Box Decides	3
Adding Two Signed Bytes	4
Where the Value Changed	4
Signed Overflow Is Not Wrapping	4
Ask Before You Add	5
The Midpoint That Overflowed	5
Run It: A Negative Array Index	5
<code>INT_MIN</code> Has No Twin	6
Multiplication Needs the Same Care	6
The Allocation That Wraps	6
Division	7
Shifting a Negative Is Not Dividing It	7
Signed and Unsigned: Do Not Guess	7
Run It: The Check That Let It Through	8
<code>&&</code> Is Not <code>&</code>	8
Subtraction Has No Negative to Land On	8
Find It Before It Finds You	8
AI Systems Connection	9
Run It: The Accumulator That Wrapped	9
Practice	9
Practice II	10
Practice III	10
Reading	10

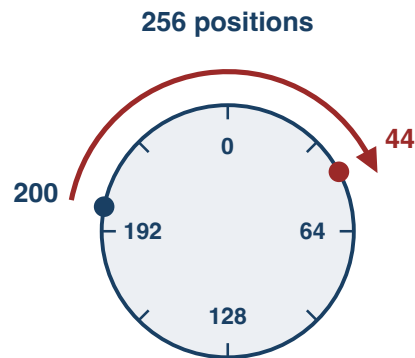
Overflow Is a Clock, Not a Crash

Same rule, different number of positions



$$9 + 5 = 2$$

not 14, and nobody is surprised



$$200 + 100 = 44$$

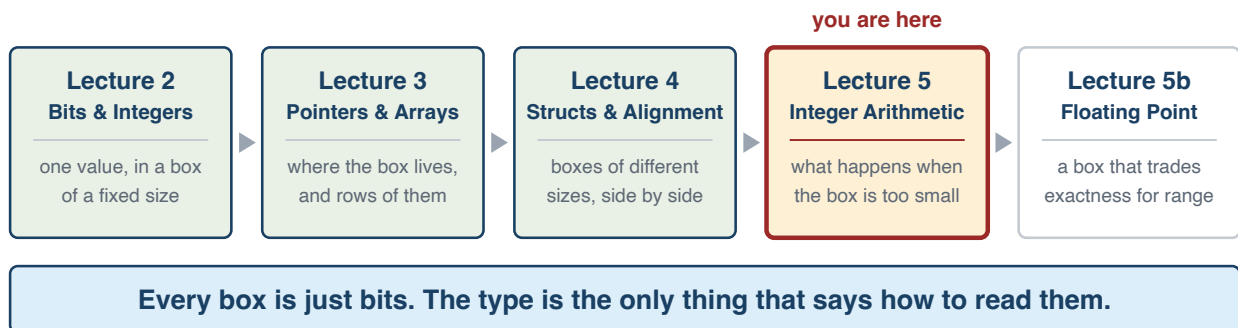
$300 - 256 = 44$, same rule

```
uint8_t a = 200, b = 100;
uint8_t c = a + b;           // 44
```

You have never once been surprised that 9 o'clock plus five hours is 2. A `uint8_t` is the same clock with 256 positions. For N-bit unsigned integers arithmetic is **modulo 2^N** — defined behaviour you can rely on, not a bug.

Where We Are

Five lectures, one idea: a value is bits in a box



Two weeks ago we filled in one box; since then, where boxes live and how several sit side by side. Today we come back to the single box and do **arithmetic** in it.

Rewind: The Box From Lecture 2

Nothing in memory says whether a byte is signed. The bits are the bits; the **type** decides how the top bit is read — as one more power of two, or as a minus sign.

bits	uint8	int8
0000 0000	0	0
0000 0001	1	1

0111 1111	127	127	
1000 0000	128	-128	<- the top bit changes sides here
1000 0001	129	-127	
1100 1000	200	-56	
1111 1111	255	-1	

Same eight patterns, two readings — Lecture 2's *One Byte, Two Readings*. Today: what happens when you add to them.

The Top Bit Is Worth -128

One column changes sign. That is the whole trick.

as uint8	128	64	32	16	8	4	2	1
	1	0	0	0	0	0	0	1
as int8	-128	64	32	16	8	4	2	1
	128 + 1 = 129							
	-128 + 1 = -127							

read as an unsigned byte

read as a signed byte

Every other column keeps its usual value. Only the top one is negative — which is why the top bit looks like a sign.

```
-2^7 + 0(2^6) + 0(2^5) + 0(2^4) + 0(2^3) + 0(2^2) + 0(2^1) + 1(2^0)
= -128 + 1 = -127
```

To *write* -127: take +127 (0111 1111), flip every bit (1000 0000), add one (1000 0001). Flip-and-add-one is a shortcut for that sum, not a separate rule.

The Odometer Rolls Over

```
255 + 1 as uint8_t -> 0    defined: unsigned wraps
127 + 1 as int8_t  -> -128  the add was fine; the squeeze back was not
```

These two lines are **not** the same story, and telling the three cases apart is most of today:

- **Unsigned wraps, and that is a promise.** Modulo 2^N , every time.
- **A narrow signed type is a *conversion* problem.** `int8_t` promotes to `int`, so `127 + 1` really is 128 and the addition never overflowed. Putting 128 back into one byte is an out-of-range conversion, which C calls **implementation-defined** — ours gives -128, and has to say so in writing.
- **`INT_MAX + 1` is a different animal.** No conversion anywhere: the arithmetic *itself* leaves the range of `int`, and that is **undefined**. Nothing is promised — including the test you wrote to catch it. Five minutes from now.

The Size of the Box Decides

Same two numbers, same addition, two different boxes to put it in:

```
uint8_t a = 200, b = 100;

sizeof(a + b);           // 4 — both operands promote to int first
```

```

a + b;                // 300 – so the sum is exact, every time

uint8_t small = a + b; // 44 – 300 does not fit in one byte
uint16_t big  = a + b; // 300 – 300 fits in two

```

The addition is **identical** on both lines. Narrow types are promoted to `int` before any arithmetic happens (C17 §6.3.1.1), so `a + b` is the `int` 300 either way. What differs is the **assignment** — and that is where a value gets lost. A large share of the integer bugs in real C code are this, dressed up.

Adding Two Signed Bytes

```

int8_t a = 100, b = 50;
int8_t c = a + b; // not 150

```

int8_t c = a + b; with a = 100, b = 50

1. C adds them as int

150

150 fits easily in a 32-bit int



2. Keep only the low 8 bits

10010110

an `int8_t` has room for 8



3. Read those bits as `int8_t`

10010110

a leading 1 means negative

= -106

Nothing overflowed while adding. The value changed at step 3, squeezing back into 8 bits — C calls that conversion implementation-defined, and our machines give -106.

Where the Value Changed

Same shape as the unsigned case: promotion first, so `a + b` is the `int` 150, exactly. The assignment is where it goes wrong.

One thing *is* different. Squeezing 300 into a `uint8_t` is defined — modulo 256, every time. Squeezing 150 into an `int8_t` is **implementation-defined**: the compiler picks a behaviour and documents it, and ours produces -106.

Signed Overflow Is Not Wrapping

If the `int` arithmetic *itself* leaves the range, that is **undefined behavior** — a different animal, and a worse one:

```

// WRONG – and this is the classic version of the mistake:
int will_overflow(int x, int y) {
    int sum = x + y;                // the overflow already happened
    if (x > 0 && y > 0 && sum < 0) return 1; // ...so this may be dead code
    return 0;
}

```

The compiler may assume signed overflow never happens, so it is entitled to delete the test.

Ask Before You Add

Test the operands, so the overflow never happens in the first place:

```
#include <limits.h>

int will_overflow(int x, int y) {
    if (y > 0 && x > INT_MAX - y) return 1;    // x + y would exceed INT_MAX
    if (y < 0 && x < INT_MIN - y) return 1;    // x + y would fall below INT_MIN
    return 0;
}
```

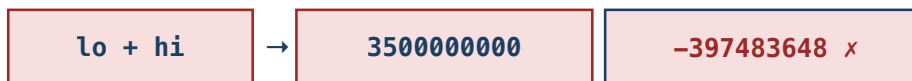
Subtraction is the mirror: $x - y$ falls below `INT_MIN` when $y > 0$ && $x < \text{INT_MIN} + y$. Or let the compiler check, as production code does — there is one of these per operator:

```
int sum;
if (__builtin_add_overflow(x, y, &sum)) { /* also _sub_ and _mul_ */
    /* handle it */
}
```

The Midpoint That Overflowed

lo = 1,500,000,000 · hi = 2,000,000,000 · INT_MAX = 2,147,483,647

(lo + hi) / 2



past `INT_MAX` — undefined, and here it wrapped

lo + (hi - lo) / 2



the difference is small, so nothing ever leaves the range

```
int mid = (lo + hi) / 2;    // the obvious one
int mid = lo + (hi - lo) / 2; // the one that works
```

This exact line was in Java's standard library, and in *Programming Pearls*, until someone noticed in 2006.

Run It: A Negative Array Index

```
lo = 1500000000    hi = 2000000000
the true midpoint is 1750000000

(lo + hi) / 2      -397483648    <- lo + hi overflowed
lo + (hi - lo) / 2 1750000000    <- hi - lo is small, so nothing overflows
```

Your binary search then reads `a[-397483648]` — and since the overflow was undefined, that number is only what this compiler did today. The average of two values in range is always in range; their **sum** is not. Never add two large values you are only going to divide again.

INT_MIN Has No Twin

Two's complement is not symmetric. There is one more negative value than positive, so `INT_MIN` has no positive counterpart:

```
INT_MIN  -2147483648
INT_MAX   2147483647
|INT_MIN| would be 2147483648, which is one past INT_MAX
```

All three of these are **undefined** on a 32-bit `int`:

```
-INT_MIN      abs(INT_MIN)      INT_MIN / -1
```

The last one usually *traps* rather than wrapping — the divide instruction raises an exception, and your program dies on the spot.

Tip

Flip the bits of `0x80000000` and add one and you get `0x80000000` back. The pattern is its own negation, which is the whole reason.

Multiplication Needs the Same Care

```
int32_t a = 100000, b = 100000;
int32_t c = a * b;           // signed overflow: undefined behavior
```

Widening the *destination* does not help — the multiply has already happened by then. Widen an **operand**, and the usual arithmetic conversions bring the other one up to match:

```
int64_t bad  = a * b;         // a * b is still computed in 32 bits
int64_t good = (int64_t)a * b; // now the multiply itself is 64-bit
```

```
int64_t bad  = a * b;         -> 1410065408
int64_t good = (int64_t)a * b; -> 10000000000
```

Note

Multiplication by a constant gets rewritten into shifts where that is cheaper. On x86-64, `x * 14` compiles to `(x << 4) - (x << 1)` — two instructions and no multiply at all. Write the multiply and let it optimise; you will not beat it.

The Allocation That Wraps

```
void *p = malloc(n * size);    // n * size is size_t arithmetic, and it wraps
```

```
n          4611686018427387904
n * 8 wraps to 0 bytes
```

```
alloc_bad -> a pointer to 0 bytes, and no complaint
alloc_ok  -> NULL, correctly
```

The loop that fills the block then writes past the end of it. This is one of the most common shapes of remote code execution in C. Check first — division cannot overflow:

```
if (size != 0 && n > SIZE_MAX / size) return NULL;
```

`calloc(n, size)` does exactly this check for you. That is the difference between `calloc(n, size)` and `malloc(n * size)`.

Division

Integer division **always rounds toward zero**:

```
5 / 2    // 2, not 3
-5 / 2   // -2, not -3
```

Note

The remainder follows from that choice: $-5 \% 2$ is -1 , because C guarantees $a == (a / b) * b + a \% b$. So $\%$ takes the sign of the left operand, which is *not* what “mod” means in mathematics.

For non-negative values, a right shift is division by a power of two:

```
x >> 3    // same as x / 8, for x >= 0
```

Shifting a Negative Is Not Dividing It

Arithmetic right shift rounds *down*; $/$ rounds toward *zero*. For negatives those disagree:

	$-7 / 2$	$-7 \gg 1$
result	-3	-4
rounded	toward zero	down

They agree only when the division is exact. To divide a possibly-negative x by 2^k with a shift, bias it first:

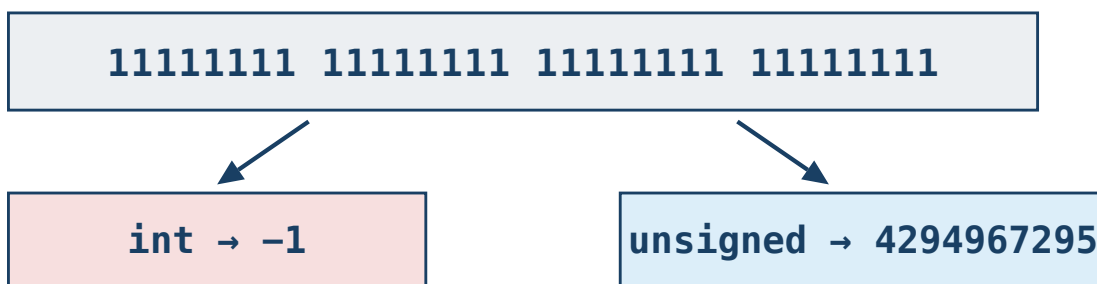
```
(x + (1 << k) - 1) >> k    // for x < 0; (-7 + 1) >> 1 == -3
```

That correction is a **bias**, and it is what the compiler emits for $x / 2$.

Use unsigned types for bit work and you avoid the rest: $10 \ll 31$ is fine, $1 \ll 31$ is undefined because 1 is a signed `int`, and any shift count at or past the type’s width is undefined too.

Signed and Unsigned: Do Not Guess

The same 32 bits, two different readings



Compare an `int` with an `unsigned` and the `int` is converted to **unsigned** first.

`int x = -1; unsigned y = 1; x < y` is false

```
int x = -1; unsigned y = 1;
x < y      // false - x is converted to unsigned, so it is UINT_MAX
-1L < 1U   // true  - a long can represent every unsigned int value
```

Which way the conversion goes depends on **rank** and on whether the signed type can hold every value of the unsigned one, so check the exact types — or better, do not mix them unless the conversion is deliberate (C17

§6.3.1.8).

Run It: The Check That Let It Through

The bounds check is signed; the use is unsigned. A negative length sails past the guard and then becomes enormous:

```
if (n > (int)cap) return;      // n is int: -1 > 10 is false
memcpy(dst, src, n);          // n becomes size_t here
```

```
copy_into(cap = 10, n = 20)    rejected: 20 is more than 10
copy_into(cap = 10, n = -1)    accepted: memcpy would be asked for
                                18446744073709551615 bytes
```

-Wall turns on -Wsign-compare, which refuses to compile the comparison in the first place. The sample file has to switch that warning **off** to show you the bug.

&& Is Not &

The logical operators ask *true or false*; the bitwise ones work column by column. They are different operators that happen to look alike:

```
1 && 2      // 1 – both operands are "true"
1 & 2       // 0 – 0b01 & 0b10 has no column with two 1s

!5          // 0 – logical NOT: 5 is true, so this is false
~5          // -6 – every bit flipped
```

&& and || also **short-circuit**: the right operand is not evaluated if the left already decides the answer. That is what makes this safe —

```
if (p != NULL && *p == 42)    // *p is never touched when p is NULL
```

— and what makes the same line with a single & a crash waiting to happen.

Subtraction Has No Negative to Land On

size_t is unsigned, so a subtraction that goes below zero comes out at the *top* of the range instead:

```
size_t capacity = 16, used = 20;
size_t remaining = capacity - used;
```

```
capacity - used = 18446744073709551612
remaining > 0   -> true
```

Hand that to memcpy and it is alloc_bad again with a minus sign instead of a multiply. **Compare before you subtract**, never after:

```
if (used < capacity)
    remaining = capacity - used;
```

The same trap eats the obvious countdown loop, where `i >= 0` is always true — Practice 2 asks you to fix it.

Find It Before It Finds You

Signed overflow is undefined, so nothing is required to tell you it happened. Ask the compiler to check at run time:

```
gcc -std=c17 -O2 -fsanitize=undefined prog.c -o prog
```

```
09-ubsan.c:26:17: runtime error: signed integer overflow:
2147483647 + 1 cannot be represented in type 'int'
```

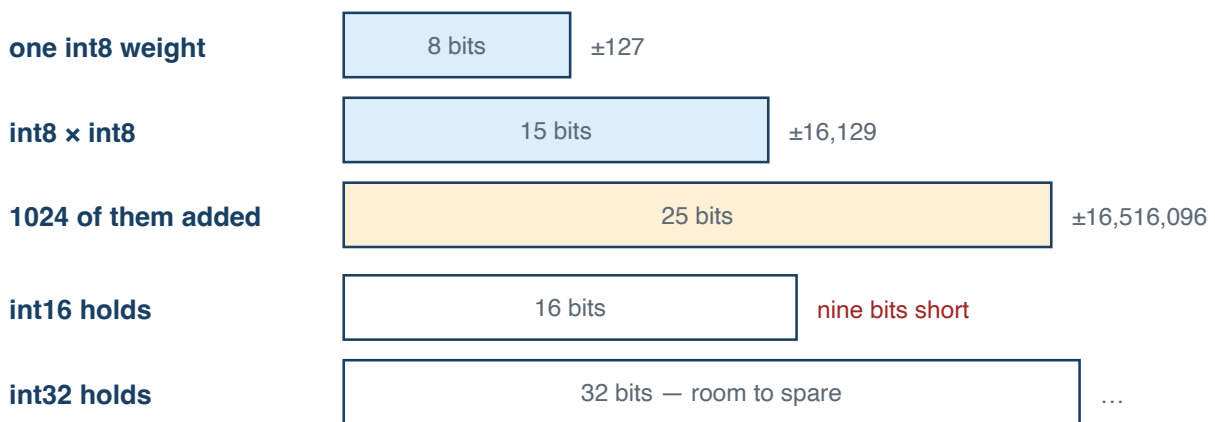
File, line, and what went wrong. It costs speed, so it is a debugging build rather than a shipping one — but turn it on for every lab you hand in and it will find the bug before the grader does.

Tip

`-fsanitize=address` is its companion: it catches reads and writes past the end of an array, and `use-after-free`. Both are worth the habit.

AI Systems Connection

Eight-bit inputs, twenty-five-bit answer



Every int8 matmul kernel on every GPU does exactly this: multiply narrow, accumulate wide.

Quantized inference stores weights and activations as `int8` — a quarter of the memory and a quarter of the bandwidth. The arithmetic cannot stay that narrow: one product already needs 15 bits, and a row of a thousand needs 25.

Run It: The Accumulator That Wrapped

```
narrow = (int16_t)(narrow + (int16_t)w[i] * a[i]); // wraps
wide += (int32_t)w[i] * a[i]; // fits
```

```
one product 127*127    16129  fits in int16
1024 of them, worst case 16516096  needs 25 bits with the sign
```

```
accumulated in int16_t 16384  <- wrapped
accumulated in int32_t 10240000
```

Multiply narrow, accumulate wide. Every `int8` matmul kernel on every GPU does exactly this, and the answer is quietly wrong if it does not.

Practice

Do today's first slide by hand, in binary. Add 200 and 100 as eight-bit columns, carrying the way you would in decimal:

```
  11001000  200
+ 01100100  100
```

```

-----
100101100  300  <- the sum needs nine bits
00101100   44  <- only eight are kept

```

Where exactly did the 256 go? Now do $255 + 1$ the same way, and then $127 + 1$ as an `int8_t`.

Practice II

1. Write a function that detects overflow in $x + y$ *without* computing the sum.
2. Why does `for (unsigned i = n - 1; i >= 0; i--)` never terminate? Rewrite it so it visits every index down to 0 and stops.
3. A buffer holds `n` items of `sz` bytes each. Write the allocation-size check so that the check itself cannot overflow.
4. What does $-5 / 2$ give in C, and what does $-5 \% 2$ give? Show that the two answers satisfy $a == (a / b) * b + a \% b$.
5. Write the binary-search midpoint so that it is correct for every pair of non-negative `int` values. Explain why yours cannot overflow.

Practice III

6. `abs(INT_MIN)` is undefined. Write a function that returns the absolute value of any `int` into a `long long`, correctly.
7. `<limits.h>` and `<stdint.h>` give you `INT_MAX`, `INT_MIN`, `UINT_MAX`, `SIZE_MAX` and the fixed-width forms. Print all five. Why is typing the digits yourself a portability bug?
8. Build `09-ubsan.c` with and without `-fsanitize=undefined`. What does each one print for `2147483647 - 1`? Which output would you rather get at 3 a.m.?

Reading

- CS:APP §2.3 (Integer Arithmetic)
- CS:APP §2.2 (Integer Representations)

Chapter and verse, if you want it. Every rule in today's lecture is one of these four, and the free N1570 draft of C11 uses the same numbering:

Clause	What it says
§6.3.1.1p2	the integer promotions : anything narrower than <code>int</code> becomes <code>int</code>
§6.3.1.8	the usual arithmetic conversions — which begin by promoting both operands
§6.3.1.3p2–3	out-of-range conversion : modular for unsigned, implementation-defined for signed
§6.5p5	overflow <i>during the arithmetic itself</i> is undefined